



Insecure deserialization

I. Serialization và Deserialization

Trong lập trình và ứng dụng phần mềm, chúng ta có thể sử dụng nhiều loại ngôn ngữ lập trình khác nhau, trong mỗi ngôn ngữ đó lại chứa các đối tượng và cấu trúc dữ liệu đa dạng. Với một lượng lớn thông tin dữ liệu với đặc điểm "không hệ thống nhất" như vậy khiến chúng ta gặp nhiều khó khăn trong các quá trình lưu trữ, truyền tải và bảo mật dữ liệu. Đó là lý do thuật ngữ "Serialization" ra đời.

Kỹ thuật Serialize được sinh ra để giải quyết vấn đề thống nhất đó. Dưới đây là một số lợi ích của việc sử dụng Serialization:

- Lưu trữ dữ liệu: Một trong những lợi ích quan trọng nhất của serialization là cho phép lưu trữ dữ liệu. Khi cần lưu trữ trạng thái của một đối tượng hoặc cấu trúc dữ liệu để sử dụng sau này, serialization cho phép chúng ta lưu trữ dữ liệu đó dưới dạng file hoặc cơ sở dữ liệu. Ví dụ, khi một ứng dụng đóng lại, serialization có thể được sử dụng để lưu trữ trạng thái của ứng dụng để có thể khôi phục lại khi ứng dụng được mở lại.
- Truyền tải dữ liệu: Serialization cũng cho phép chuyển đổi đối tượng hoặc cấu trúc dữ liệu thành dạng có thể truyền tải qua mạng. Khi cần truyền tải dữ liệu từ một ứng dụng đến ứng dụng khác qua mạng, serialization giúp chúng ta chuyển đổi đối tượng hoặc cấu trúc dữ liệu thành dạng có thể truyền tải. Ví dụ, khi một ứng dụng phải gửi một đối tượng qua mạng tới một ứng dụng khác để xử lý, serialization có thể được sử dụng để chuyển đổi đối tượng thành một chuỗi byte nhằm phù hợp việc truyền tải.
- Tương tác giữa các ngôn ngữ lập trình khác nhau: Serialization cho phép tương tác giữa các ngôn ngữ lập trình khác nhau. Khi các ứng dụng được phát triển bằng các ngôn ngữ lập trình khác nhau và cần truyền tải hoặc chia sẻ dữ liệu, serialization cho phép chuyển đổi dữ liệu sang dạng chung để có thể được sử



dụng bởi tất cả các ứng dụng. Ví dụ, một ứng dụng được viết bằng Java có thể chuyển đổi đối tượng thành chuỗi byte và gửi nó đến ứng dụng được viết bằng Python hoặc C#.

- Bảo mật dữ liệu: Serialization cho phép bảo mật dữ liệu. Khi dữ liệu được chuyển đổi sang dạng chuỗi byte, chúng ta có thể mã hóa nó để bảo mật dữ liệu.

Thông thường, dữ liệu bytes được chọn làm quy tắc chung nhằm mang lại hiệu quả tương tác tốt nhất với nguyên tắc I/O (Input/Output) trong quá trình lưu trữ và truyền tải dữ liệu.

Ngược lại với Serialize, kỹ thuật Deserialize giúp ứng dụng có thể chuyển các chuỗi byte đó trở lại thành đối tượng hoặc cấu trúc dữ liệu gốc.

Để hiểu rõ hơn về hai quá trình này, các bạn có thể hình dung công việc phải vận chuyển một tòa nhà từ vị trí này sang vị trí khác: Serialization là quá trình tháo dỡ tòa nhà thành từng viên gạch, và tạo ra một bản thiết kế thi công; Sau khi chuyển các viên gạch tới vị trí đích, quá trình Deserialization sẽ khôi phục lại tòa nhà từ bản thiết kế đó.

II. Một số phương thức thực hiện Serialize - Deserialize

Về bản chất thì Serialization - Deserialization là quá trình "phân mảnh - tái tổ chức" một đối tượng nên không có một quy tắc chung để thực hiện nó. Chúng có thể được chia làm 3 hình thức chính sau: binary serialization, SOAP (Simple Object Access Protocol) serialization, và XML (Extensible Markup Language) serialization. Trong đó:

- Binary Serialization là quá trình chuyển đổi các đối tượng trong bộ nhớ thành một chuỗi các byte. Chuỗi này có thể được sử dụng để lưu trữ hoặc truyền qua mạng. Binary Serialization là định dạng hiệu quả và nhanh nhất trong số các phương pháp serialization, vì nó chỉ tạo ra một chuỗi byte duy nhất. Tuy nhiên,



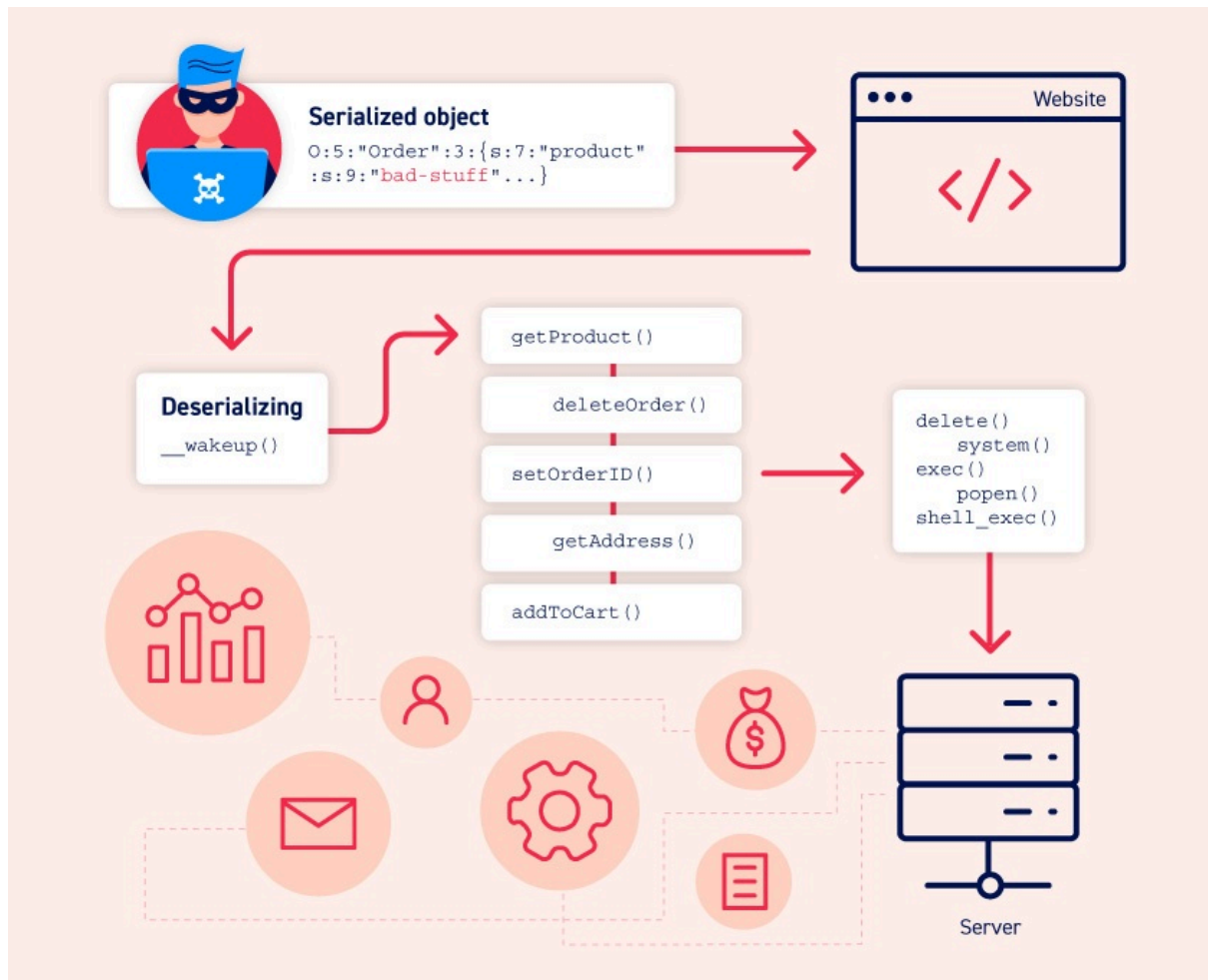
định dạng này có thể không tương thích giữa các nền tảng ngôn ngữ, công nghệ khác nhau hoặc các phiên bản khác nhau của chương trình.

- SOAP (Simple Object Access Protocol) Serialization là một phương thức serialization được sử dụng trong web service để truyền tải các thông tin giữa các ứng dụng khác nhau. SOAP Serialization sử dụng định dạng XML để tạo ra các tin nhắn trao đổi giữa các ứng dụng. SOAP Serialization có thể được sử dụng để gửi các yêu cầu (requests) và nhận các phản hồi (responses) từ các dịch vụ web.
- XML Serialization là quá trình chuyển đổi các đối tượng trong bộ nhớ thành một định dạng XML. Định dạng này có thể được sử dụng để lưu trữ hoặc truyền qua mạng. Với hình thức lưu trữ và truyền dưới dạng văn bản làm cho dữ liệu dễ đọc và tương thích giữa các nền tảng khác nhau. Tuy nhiên, định dạng này thường có độ phức tạp cao hơn và chậm hơn so với Binary Serialization.

Ngoài ra lập trình viên có thể tự định nghĩa cách thức chuyển đổi một đối tượng trong bộ nhớ thành một định dạng có thể lưu trữ hoặc truyền qua mạng. Điều này giúp họ điều khiển được các quá trình chuyển đổi dữ liệu, bao gồm cách mã hóa đối tượng và cách giải mã đối tượng. Cách làm này thường được gọi là Custom Serialization.

III. Lỗ hổng Insecure deserialization

Insecure deserialization là lỗ hổng bảo mật xảy ra khi dữ liệu do người dùng kiểm soát được deserialized mà không có các biện pháp an toàn, cho phép kẻ tấn công thao túng dữ liệu này để tiêm mã độc hại vào ứng dụng. Quá trình *serialization* chuyển đổi các cấu trúc dữ liệu phức tạp thành một chuỗi byte để truyền tải, trong khi *deserialization* phục hồi dữ liệu này về trạng thái ban đầu. Việc thiếu kiểm soát an toàn trong deserialization có thể dẫn đến các cuộc tấn công nghiêm trọng như *remote code execution* (RCE), leo thang đặc quyền, truy cập tùy ý vào tệp, hoặc từ chối dịch vụ.



IV. Nguyên nhân và cách khai thác

- **Thiếu hiểu biết về nguy cơ:** Người phát triển thường coi dữ liệu deserialized là an toàn, đặc biệt là với các ngôn ngữ sử dụng định dạng nhị phân. Điều này dễ dẫn đến sự tin tưởng không đúng chỗ vào dữ liệu.
- **Hạn chế trong việc kiểm tra dữ liệu:** Các phương pháp kiểm tra dữ liệu sau khi đã deserialized thường không hiệu quả, vì mã độc có thể hoạt động ngay trong quá trình deserialization.
- **Khó khăn trong quản lý phụ thuộc:** Website thường sử dụng nhiều thư viện và lớp khác nhau, dẫn đến khó khăn trong việc kiểm soát các hành động không mong muốn có thể xảy ra.

V. Phòng chống lỗ hổng

- **Tránh deserialization dữ liệu không tin cậy:** Tốt nhất là không nên deserialize dữ liệu từ nguồn không tin cậy.



- **Áp dụng chữ ký số:** Xác thực tính toàn vẹn của dữ liệu trước khi deserialization.
- **Tạo phương thức serialization riêng:** Thay vì sử dụng các tính năng deserialization chung, nên tạo phương thức tùy chỉnh để kiểm soát trường dữ liệu nào được expose.
- **Tránh phụ thuộc vào gadget chains:** Tìm cách loại bỏ tất cả gadget chains là không khả thi; do đó, giải pháp là hạn chế tối đa việc deserialization từ nguồn không đáng tin cậy.

VI. Tham khảo

<https://viblo.asia/p/insecure-deserialization-vulnerability-cac-lo-hong-insecure-deserialization-phan-1-EvbLb5pPJnk>

<https://portswigger.net/web-security/deserialization>

Mọi thắc mắc hay ý kiến đóng góp vui lòng liên hệ :

Email : r3dctf.info@gmail.com

@Design by R3D CTF Team